

Jackson Crist

CSCI 499

**Senior Project
Implementation/Defense**

Interstella Synth VST

B.S. In Computer Science

Contents:

Introduction - Page 2

About The Project - Page 2

Project Goals - Page 3

Tools Used - Page 3

Design - Page 3

Testing - Page 5

Issues/Challenges - Page 5

Future Goals - Page 5

Introduction:

I am incredibly grateful for this opportunity to share my senior project with all of you. It was very exciting to be able to combine the skills that I have learned in my studies here with something that I have a great passion for. I would like to thank Dr. West for his guidance and his encouragement as my adviser for this project, and to thank all of my teachers so far that have helped me to further my education.

About My Project:

My project is a virtual synthesizer plug-in using the VST framework, which for those of you who may not be familiar with this term, it stands for Virtual Studio Technology and was released by Steinberg in 1996. It is an audio plug-in interface for use with Digital Audio Workstation (DAW) programs like FL Studio, Ableton, or Cubase. VSTs use DSP, or Digital Signal Processing, to simulate synthesizer/effects hardware, like a synth or compressor.

The sound for the synthesizer is made by soundwaves that are produced by a DSP-generated oscillator which generates up to 8 simultaneous voices using one of five waveforms: Sine, Square, Triangle, Saw, and a white noise generator.

Once the sound is produced, the envelope modulates the volume using four different parameters. The attack controls how fast the sound comes in, decay controls how fast the volume drops to the sustain level, the sustain sets the level that the note stays at while being held, and the release controls how long it takes for the note to fade after being let go.

So the oscillator produces the sound, the envelope adjusts the volume level, and the filter controls which frequencies are played. The low pass filter only allows frequencies below a certain threshold. The high pass filter only allows frequencies above a certain threshold. The band pass filter only allows frequencies in a certain range. The cutoff sets the threshold to block certain frequencies. The resonance boosts frequencies at the cutoff level.

The previous three components are essential to any decent synthesizer. The next two I'm about to show you are simply elements that I find useful and wanted to include. First among these is the LFO. An LFO is a Low Frequency Oscillator, which is a sine wave oscillator that can be set to modulate different parameters. My LFOs can control either pitch, volume, or filter cutoff. The depth sets how much the parameter is changed, and the frequency controls the speed of the change.

The second additional feature I wanted to add was an arpeggiator, which plays the individual notes of a chord one by one in a designated pattern. My arpeggiator can play notes in an ascending, descending, or up and down alternating pattern. The speed at which these notes are played is able to be controlled.

Project Goals:

I decided to pursue this project for three main reasons. I wanted a synthesizer I could use myself that had specific features that I like, I wanted to combine my love of music and music production with what I've learned here, and I wanted to become more familiar with the tools used to make synth applications so I could continue to do this in the future.

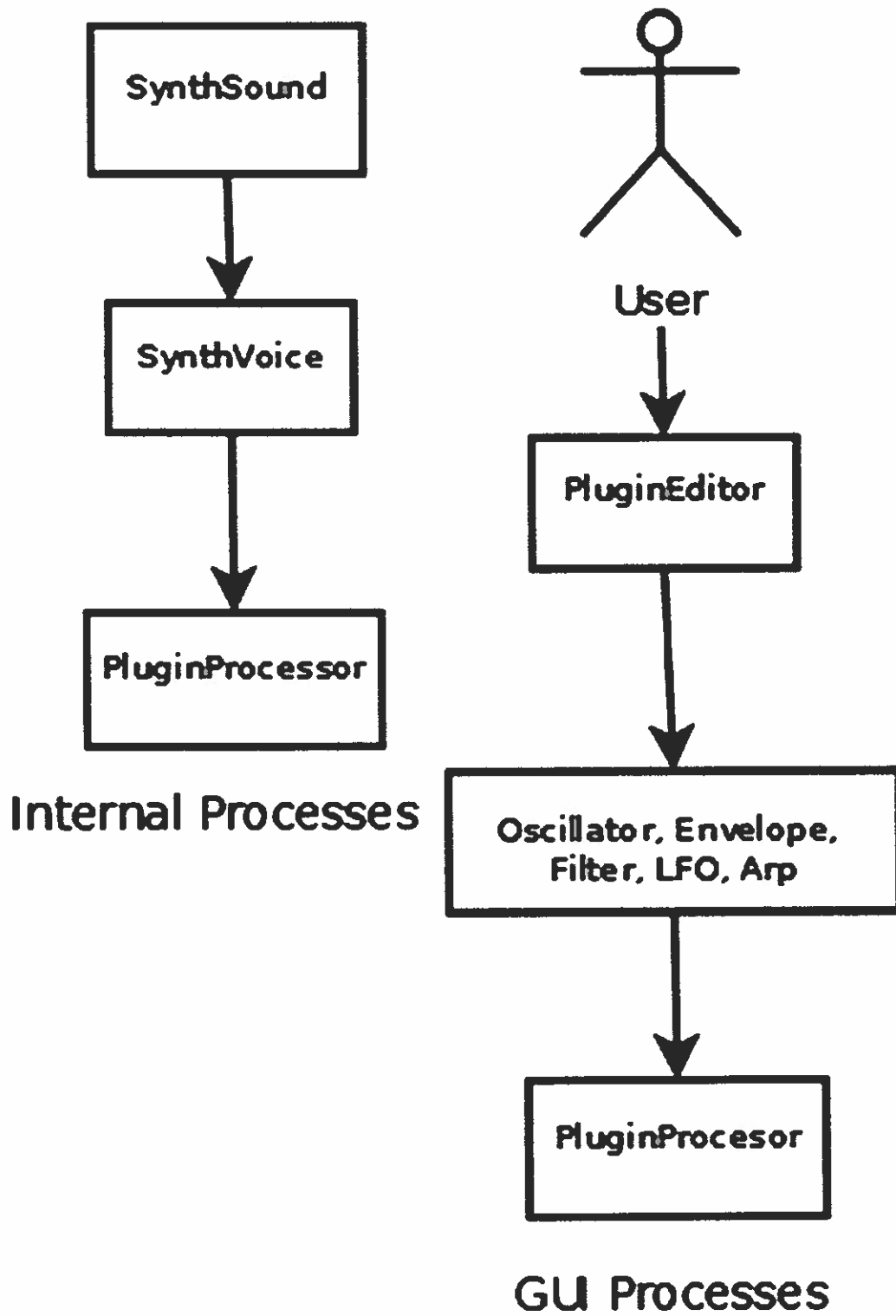
Tools Used:

These tools that I found and decided to work with were C++ libraries, so the program was made with C++. The majority of the coding was done in Visual Studio, although some was done with JUCE's Projucer editor. JUCE is the library that provides the setup for the VST framework, and the oscillators and DSP were done with Maximilian, which is a free, open-source DSP library.

Design:

The design of my program has the SynthSound enable the sound to be played, SynthVoice applies the effects, and the PluginProcessor puts all the components and information the plug-in receives together. The user interfaces with the GUI which is generated by the PluginEditor stitching

together all the individual designs that the effects classes create, and passes the info along to the processor.



Testing:

The testing was done by creating each component one at a time and testing manually in the JUCE Plugin Host. Occasionally, if certain portions were giving any issue, I would set breakpoints at certain lines using the Visual Studio debugger to see what the program was doing at those points.

Issues/Challenges:

Naturally, I encountered a few problems while working on this. I had no experience working with digital signal processing, so learning from scratch was a challenge. The LFO class was the most difficult to figure out and was the last piece to be implemented. There are still minor issues, such as the tuning being slightly off, and the fact that the sound is not quite "robust" enough for my preferences.

Future Goals:

Now that I have finally completed this project, I've come away from it with new knowledge and a brand new set of goals for the future. I am much more familiar with DSP programming now, and am much more prepared to make other, hopefully more powerful, plug-ins. Now, I would like to fix some of the issues I have encountered and possibly add more functions. I would also like to find other, more professional, DSP libraries to improve the sound in my next applications.

My code can be found at: <https://github.com/jacksoncris94/SeniorProject>

INTERSTELLA SYNTH VST

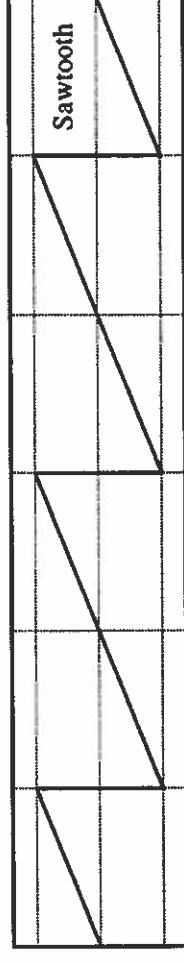
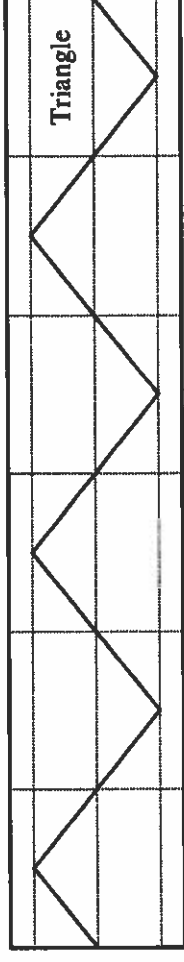
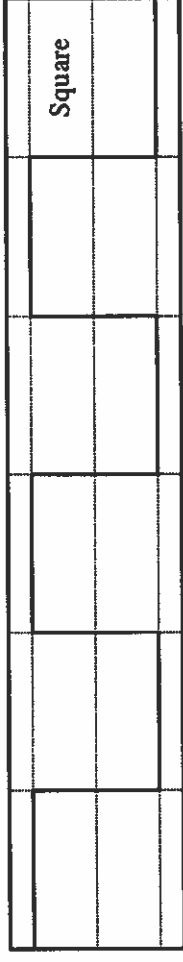
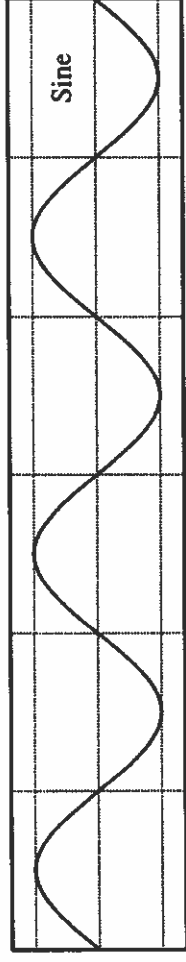
What is a VST?

- .Virtual Studio Technology**
- .Released by Steinberg in 1996**
- .Audio plug-in interface for use with Digital Audio Workstation (DAW) programs**
- .Uses DSP (digital signal processing) to simulate synthesizer/effects hardware**

What can this synthesizer do?

.Oscillator

- Generates up to 8**
- simultaneous voices**
- using one of five**
- waveforms**
- Sine, Square, Triangle,**
- Saw, and Noise**



What can this synthesizer do? (Cont.)

.Envelope

- Attack: How fast the sound comes in**
- Decay: How fast the volume drops to the sustain level**
- Sustain: The level that the note stays at while being held**
- Release: How long it takes for the note to fade after being let go**

What can this synthesizer do? (Cont.)

.Filter

-Type

.Low pass: Only allows frequencies below a certain threshold

.High pass: Only allows frequencies above a certain threshold

.Band pass: Only allows frequencies in a certain range

-Cutoff: Sets the threshold to block certain frequencies

What can this synthesizer do? (Cont.)

- **LFO (not the boy band)**

- **Low Frequency Oscillator**

- **Sine wave oscillator that modulates frequency of specified parameter**

- **Modifiable parameters: Pitch, Volume, Filter Cutoff**

- **Depth: How much parameter is modified**

- **Frequency: How fast parameter is changed**

What can this synthesizer do? (Cont.)

.Arpeggiator

- Plays the notes of a chord one by one in a pattern**
- Plays notes: Up, Down, and Alternating**
- Speed can be controlled**

Project Goals

- .To make a synthesizer with specific features**
- .To combine my love of music with my field of study**
- .To become more acquainted with DSP programming and with the tools I used**

Tools

.C++

- Visual Studio**
- JUCE and Projucer**
- Maximilian DSP library**

Design

.SynthVoice does the heavy

.lifting DSP-wise

.PluginProcessor gets

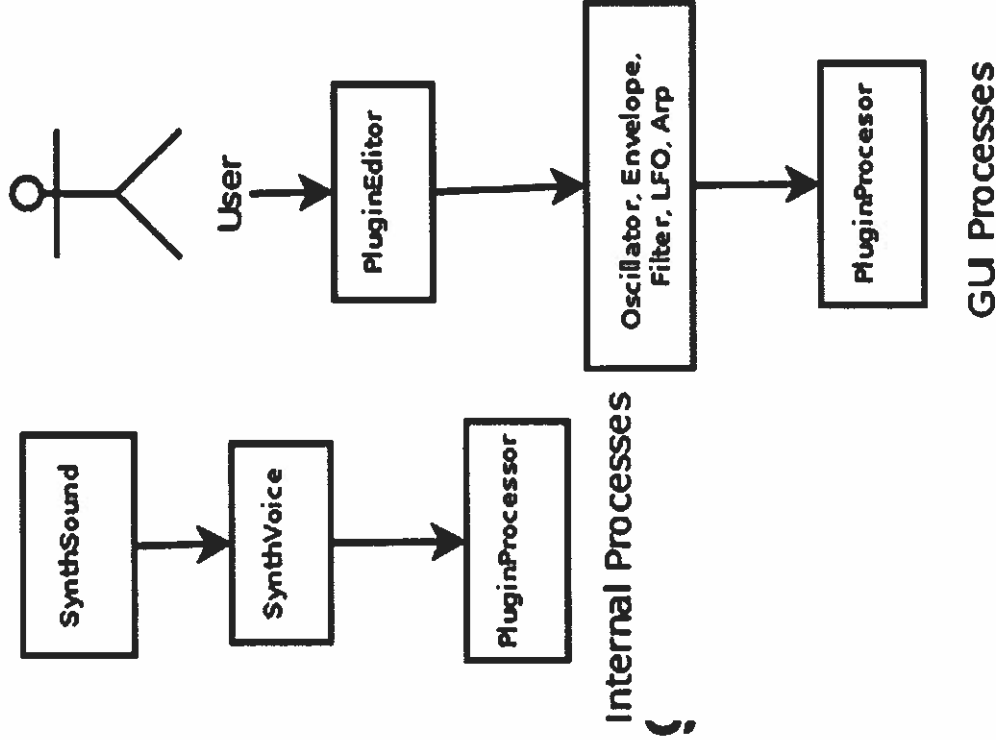
.values from controls,

.combines with MIDI (Musical

.Instrument Digital Interface) messa

.Effects classes make controls

.that PluginEditor draws



Testing

- .Construction was done one element at a time**
- .Majority of testing was done with JUCE plugin host**
- .Additional debugging was done with Visual Studio debugger**

Issues

- .Learning DSP functions was a hurdle**
- .LFO class was the last to be finished**
- .Still some minor problems; tuning not quite right**
- .Not quite satisfied with sound of DSP library; not “robust” enough for my preferences**

Findings/Future Goals

- .Much more familiar with technology and DSP**
- .Would like to implement more functions/improve on the ones already made**
- .Interested in finding new DSP libraries to use in other projects**

Sources

.TheAudioProgrammer: Juce Tutorials -

<https://www.youtube.com/watch?v=7n16Yw51xkl&list=PLLgJJsrdrwhPxa6-02-CeHW8ocwSwl2jnu>

https://docs.juce.com/master/tutorial_plugin_examples.html#tutorial_plugin_example_s_arpeggiator

<https://christianfloisand.wordpress.com/2012/04/18/coding-some-tremolo/>

.Waveform Image credit -

https://en.wikibooks.org/wiki/Sound_Synthesis_Theory/Oscillators_and_Wavetables#/media/File:Waveforms.svg

Questions?